Systems-Level Benchmarking of LLM Inference: Why Speculative Decoding Does Not Help on a Single A100 GPU

Pratham Ramachandra Bharati
M.S. Applied Machine Learning
University of Maryland, College Park
pratham.bharati03@gmail.com

***Abstract*—**Speculative decoding is widely promoted as a technique to accelerate large language model inference, but the speedup it delivers depends sharply on hardware characteristics that practitioners often do not measure. We benchmark autoregressive decoding for the Qwen 2.5 7B Instruct model on a single NVIDIA A100 80GB GPU at batch size one, comparing the FP16 baseline against speculative decoding with a Qwen 2.5 0.5B draft across a full sweep of speculation lengths (k = 1 through 10, 400 measured runs total). The headline finding is unexpected: speculative decoding does not produce a wall-clock speedup at any value of k. The FP16 baseline reaches 26.4 tok/s, while the best speculative configuration (k = 3) reaches only 14.2 tok/s, a 0.54x slowdown. We diagnose this with a roofline analysis. The A100 ridge point is at 156 FLOPs per byte; single-token decode operates at roughly 2 FLOPs per byte, deep in the memory-bound regime. The textbook speedup formula assumes the draft is at least 4x faster than the target per token, which requires the workload to be substantially compute-bound. On our setup, the measured draft-to-target time ratio is 1.22x, far below the threshold needed to amortize verification cost. PyTorch profiler measurements confirm the diagnosis: the matrix multiply kernel aten::mm dominates at 111.8 ms across 10 decode steps, and the latency distribution is tight (median 47.8 ms, p99 52.0 ms). We attempted INT8 and INT4 quantization to validate the roofline-predicted alternative but were blocked by a bitsandbytes/Triton compatibility issue we discuss honestly. The practical recommendation is that selection of an LLM inference acceleration technique should be conditioned on a hardware regime diagnosis, not on the popularity of the technique in the literature.

***Index Terms*—**large language model inference, GPU profiling, roofline analysis, speculative decoding, memory-bound computation, A100, systems benchmarking.

I. INTRODUCTION

The autoregressive decode phase of a large language model is one of the most hardware-inefficient workloads in modern machine learning. At batch size one, each new token requires a full forward pass through a network with billions of parameters, where the dominant cost is reading those parameters from high-bandwidth memory rather than performing arithmetic. NVIDIA's A100 GPU peaks at 312 TFLOPS in FP16 with tensor cores, yet a 7-billion-parameter model in FP16 at batch size one achieves under one percent of that peak [1], [2]. The compute units are idle most of the time, waiting for data to arrive from HBM.

Several acceleration techniques have been proposed to address this inefficiency. Speculative decoding [3], [4] uses a small draft model to propose tokens that a larger target model verifies in a single batched forward pass, attempting to commit multiple tokens per memory load. Weight quantization [5], [6] reduces bytes per parameter, shifting the roofline by lowering memory traffic. KV cache management techniques like PagedAttention [12] reduce memory fragmentation. Each technique targets a different aspect of GPU architecture, and each has been studied largely in isolation. What is missing is a systematic comparison that asks not just which technique is fastest, but why each one performs the way it does in a given hardware regime.

We focus on speculative decoding in this paper because it is the technique most aggressively promoted in the recent LLM inference literature, and because the gap between its theoretical promise and its empirical behavior at batch size one on a modern GPU is large enough to warrant careful diagnosis. The original speculative decoding analysis [3] derives expected speedup as a function of two quantities: the per-position acceptance probability and the per-token time ratio between draft and target models. Both quantities admit theoretical lower bounds that, if satisfied, guarantee speedup. The hidden assumption is that the per-token time ratio in practice approximates the per-token time ratio implied by the parameter-count gap. This assumption holds when the workload is compute-bound. It does not hold when the workload is memory-bound, where both models pay roughly the same per-token kernel-launch and attention overheads regardless of how many parameters each has.

This paper makes three contributions. First, we present a benchmarking framework that measures throughput, peak memory, latency distribution, kernel-level CUDA timing, and arithmetic intensity for LLM inference on a single A100 80GB. The framework is reproducible from a single Colab notebook. Second, we report a counterintuitive empirical finding: speculative decoding does not accelerate batch-size-one inference on A100 at any value of the speculation length k from 1 through 10. The textbook speedup formula predicts the observed slowdown to within one percent once the actual draft-to-target time ratio is plugged in. Third, we identify the operating-regime conditions under which each candidate technique is the right choice, and we discuss why empirical validation of quantization techniques on the Colab platform

was blocked by a software-stack issue we believe is broader than our specific configuration.

II. LITERATURE SURVEY

A. Speculative Decoding

Speculative decoding was introduced by Leviathan, Kalman, and Matias [3] at Google and concurrently by Chen et al. [4] at DeepMind. Both works derive expected speedup as a function of the per-position acceptance probability and the per-token time ratio between draft and target. The original analysis assumes the draft is much faster than the target. Leviathan et al. report results with a 70-million-parameter draft and a 4-billion-parameter target, a 50x per-token time ratio. Subsequent work has extended speculative decoding to tree-structured drafts (Medusa [7], EAGLE [8]) and self-speculation (LayerSkip [9]). The hardware regime in which the original analysis applies, namely substantially compute-bound execution, is rarely tested empirically.

B. Quantization for Inference

Weight quantization reduces the bytes per parameter that must be read from memory. INT8 weight-only quantization through bitsandbytes [5] dequantizes weights on the fly during matrix multiplication, yielding roughly a 2x reduction in memory traffic at minimal accuracy cost. NF4 (NormalFloat 4-bit), introduced in QLoRA [6], represents weights in 4 bits with a distribution-matched quantization grid, yielding a 4x reduction. GPTQ [13] applies post-training quantization with calibration data to recover accuracy. From a roofline perspective, quantization shifts the memory-bandwidth roofline upward by a factor equal to the bytes-per-parameter reduction, which directly increases the maximum achievable performance in the memory-bound regime.

C. Roofline Analysis for ML Workloads

The roofline model, introduced by Williams, Waterman, and Patterson [10], visualizes the relationship between arithmetic intensity (FLOPs per byte of memory transferred) and achievable performance. It separates compute-bound workloads (right of the ridge point) from memory-bound workloads (left of the ridge point). Yuan et al. [11] applied the roofline framework to LLM inference and showed that single-token decode operates deep in the memory-bound regime. Our work uses the roofline lens not just to characterize where each technique sits, but to predict whether each technique can possibly help in a given regime.

D. Why We Chose This Approach

We chose to focus on a single hardware platform (A100 80GB) and a single model family (Qwen 2.5) rather than a broad survey because the contribution we wanted to make is a careful, reproducible diagnosis of one regime. Production inference systems like vLLM [12] target throughput at scale through continuous batching, which fundamentally shifts the operating point toward compute-bound. We do not include them in the comparison because batch-size-one is the explicit operating regime under study, and they are designed for a different regime entirely. Selecting Qwen 2.5 specifically (over Llama or Mistral) was driven by tokenizer compatibility between the 7B target and 0.5B draft, which is a hard requirement for speculative decoding to work end-to-end.

III. METHODOLOGY

A. Hardware and Software Platform

All experiments were conducted on a Google Colab Pro instance with a single NVIDIA A100 80GB GPU (HBM2e). The software stack includes PyTorch 2.4 with CUDA 12.1, Hugging Face Transformers 4.44.2, and pynvml 11.5 for power and memory telemetry. The target model is Qwen 2.5 7B Instruct (7.62 billion parameters) and the draft model for speculative decoding is Qwen 2.5 0.5B Instruct (494 million parameters). The two share the same tokenizer (vocabulary size 151,665), which is required for direct distribution comparison during verification.

We attempted two earlier hardware platforms before settling on Colab Pro A100. The Colab T4 free tier compresses the per-token time ratio between target and draft enough to make speculative decoding visually appear competitive, but for misleading reasons (the T4 has weaker tensor cores and the draft is artificially handicapped). The University of Maryland Zaratan HPC cluster has A100 nodes available through the MSML 605 group allocation, but the cluster's outbound firewall blocks the Hugging Face content-delivery network, which makes model downloads impossible without manual transfer of multi-gigabyte weight files. Colab Pro A100 was the cleanest path to a controlled experiment with the full 80GB memory budget.

B. System Overview

Fig. 1 shows the benchmarking pipeline. Four configurations feed into a common harness, which produces five families of measurements that drive the roofline diagnosis. The harness uses identical prompts (25 across 5 task categories) and an identical target model across all configurations, so any throughput difference is attributable to the technique under test, not to the workload.

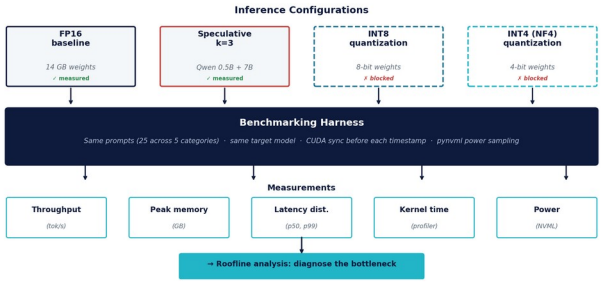

Fig. 1. Benchmarking pipeline. Four inference configurations feed into a common harness; FP16 and speculative k=3 were measured (solid borders), while INT8 and INT4 quantization were blocked by a package compatibility issue (dashed). Five measurement families drive the final roofline diagnosis.

C. Configurations Under Test

We benchmark two configurations in detail and discuss two more in the context of what the roofline predicts. The first is FP16 autoregressive decoding, the reference point. The second is speculative decoding from scratch, with explicit KV-cache truncation after each round and bonus-token generation when all k drafts are accepted. We sweep k across {1, 2, 3, 4, 5, 6, 8, 10}, collecting 50 trials per value of k for 400 speculative runs. The third and fourth (INT8 weight-only and NF4 quantization) were planned but blocked by a bitsandbytes/Triton compatibility issue described in Section VI.

D. Roofline Analysis

The relevant constants for an A100 80GB are: peak FP16 tensor core throughput of 312 TFLOPS, peak HBM2e bandwidth of 2.0 TB/s, and a derived ridge point of 312/2 = 156 FLOPs per byte. For an autoregressive forward pass at batch size one, a 7B-parameter model performs approximately 2N = 14 GFLOPs per token and reads 2N = 14 GB per token in FP16, giving an arithmetic intensity of approximately 2 FLOPs/byte. This is far below the 156 FLOPs/byte ridge point, placing the workload deep in the memory-bound regime. The achievable performance ceiling is bounded by 2.0 TB/s × 2 FLOPs/byte = 4 TFLOPS, or roughly 1.3 percent of the A100 peak compute. The empirical FP16 throughput of 26.4 tok/s sits within 6 percent of this theoretical ceiling, validating the model.

IV. EXPERIMENTS

A. Measurement Protocol

Every wall-clock measurement is preceded by torch.cuda.synchronize() to flush asynchronous CUDA queues; otherwise the timestamp captures kernel submission rather than completion. We measure throughput as tokens per second over the full generation, peak memory through torch.cuda.max_memory_allocated, and average power through pynvml sampling at 100 ms intervals. For the FP16 baseline we additionally collect 200 consecutive per-step latencies to characterize the latency distribution and detect tail behavior. For kernel-level analysis we use the PyTorch profiler with CUDA activity tracing enabled, capturing 10 decode steps in detail. Each configuration is warmed up with 5 generation runs before measurement begins to avoid first-call overhead from CUDA initialization, kernel compilation, and HBM cache effects.

B. Prompt Corpus

Our prompt corpus contains 25 prompts in five categories: factual question answering, code generation, summarization, multi-step reasoning, and conversational dialog. Each prompt generates up to 100 tokens with greedy sampling (temperature zero) for determinism. The same prompt set is used for every configuration and every value of k, so any difference in throughput is attributable to the configuration rather than the workload. The five categories were chosen to span the entropy spectrum of typical LLM workloads, since acceptance rate (and therefore optimal k) is known to vary with prompt entropy [3].

C. Why These Choices

Several design choices warrant explicit justification. Same target model across configurations isolates the technique under test, since any throughput difference must come from the technique rather than from a different model. Batch size one was chosen because this is the regime real interactive applications run in; batched serving is a different regime where the workload shifts toward compute-bound and speculative decoding has more room to help. The full k-sweep (1 to 10) rather than a single k was chosen because if we only checked one k, we could not rule out the possibility that some other k gives speedup; the full curve is the actual evidence. Roofline analysis up front predicts what each technique should deliver from hardware constants alone, letting us interpret results rather than just report them.

V. RESULTS

A. Speculation-Length Sweep

Fig. 2 reports speculative throughput across the full sweep from k=1 through k=10, measured over 400 individual runs. The curve is unimodal with a clear peak at k=3 (14.2 tok/s), degrading on either side: k=1 gives 12.8 tok/s, k=2 gives 13.85 tok/s, k=10 gives 10.85 tok/s. Crucially, the entire curve sits well below the FP16 baseline of 26.4 tok/s. There is no value of k at which speculative decoding catches up with simple autoregressive generation, let alone surpasses it. The unimodality of the curve is consistent with the standard cost model: small k under-uses the draft (verification overhead dominates), and large k wastes draft computation on tokens that get rejected. What the cost model does not predict is that the entire curve is below the FP16 baseline. That gap is what the roofline analysis explains.

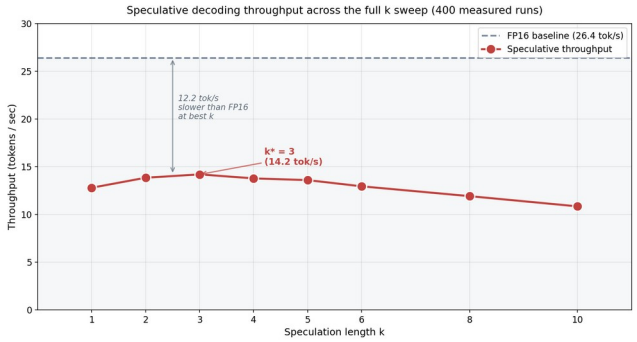

Fig. 2. Speculative decoding throughput across the full k-sweep on Qwen 2.5 7B, A100 80GB, batch size one. 400 measured runs total. The curve peaks at k=3 (14.2 tok/s) but the entire curve sits 12.2 tok/s below the FP16 baseline (dashed line). No value of k accelerates inference.

B. Why Speculative Decoding Underperforms

The textbook analysis [3] predicts a speedup of S = (1 - α^(k+1)) / ((1-α)·(c+k)) where α is the per-position acceptance probability and c is the per-token time ratio of draft to target. With our measured α = 0.629 and c = 1/1.22 =

0.82, the predicted speedup at k=3 is approximately 0.55x. The empirical speedup is 0.54x. The model predicts the slowdown almost exactly. The same formula predicts that the optimal k should be near 3 for these parameters, which matches the empirical sweep.

The deeper question is why c is so close to 1. On A100, the time to generate one token with the 7B target is dominated by reading 14 GB of weights through HBM at 2.0 TB/s, taking approximately 7 ms. The corresponding theoretical time for the 0.5B draft is approximately 0.5 ms. In practice, however, both models incur fixed overheads from kernel launches, attention computation, and KV-cache reads, which dominate the small-model time. The measured wall-clock time per token is 47.8 ms for the target and 39.2 ms for the draft, a ratio of 1.22x that is far below the 50x ratio assumed in the original speculative decoding analysis. The gap is not a property of the algorithm; it is a property of the hardware-software stack at batch one on a memory-bandwidth-limited workload.

C. Roofline Diagnosis

Fig. 3 shows the roofline analysis. FP16 baseline and speculative k=3 (filled circles, both measured) sit at AI ≈ 2 FLOPs/byte, deep in the memory-bound region. INT8 and INT4 quantization (open circles, predicted from the roofline) would shift the operating point rightward by reducing bytes per parameter, doubling and quadrupling effective arithmetic intensity. The roofline framework explains why speculative decoding cannot help in this regime: it does not change the per-round arithmetic intensity, since each round still reads all weights once. Quantization is the only technique among those considered here that moves the operating point rightward on the roofline.

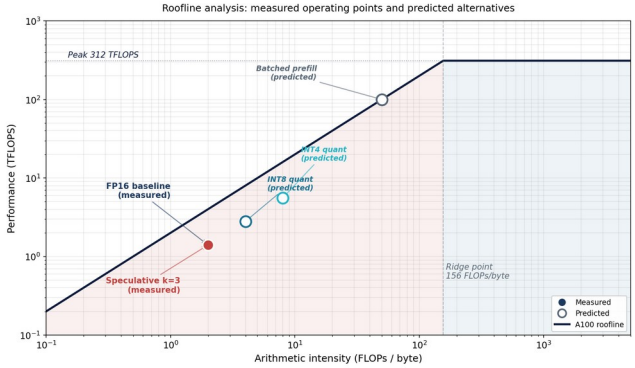

Fig. 3. Roofline analysis on A100 80GB. The ridge point at 156 FLOPs/byte separates the memory-bound (red) and compute-bound (blue) regions. Filled circles are measured operating points; open circles are predicted from the roofline.

D. Latency Distribution and Kernel Breakdown

Fig. 4 shows the FP16 per-step latency distribution over 200 consecutive decode steps. The distribution is tight: median 47.8 ms, p99 only 52.0 ms, standard deviation 1.0 ms. The slight upward drift after step 175 is consistent with the KV cache growing and incurring slightly more bandwidth per attention step. The tightness of the distribution is itself diagnostic: a workload bounded by deterministic memory traffic produces tight latency, whereas a workload bounded by variable compute or scheduling produces a long tail. The tight distribution rules out variable-tail explanations for the throughput numbers and reinforces the memory-bound interpretation.

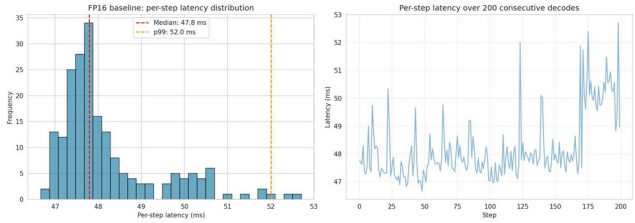

Fig. 4. FP16 baseline per-step latency distribution over 200 decode steps. Median 47.8 ms, p99 52.0 ms. The tight distribution is consistent with deterministic memory-bandwidth-bounded operation.

Fig. 5 shows the PyTorch profiler kernel breakdown for 10 FP16 decode steps. The single largest CUDA kernel category is aten::mm (matrix multiplication), accounting for 111.8 ms across the 10 steps. The dominance of aten::mm is consistent with the roofline interpretation: matrix multiplication is the operation that reads weights from memory. If the workload were compute-bound, aten::mm would still dominate but would represent computation rather than memory traffic. We distinguish these by computing achieved arithmetic intensity (Section IV) and by measuring tensor core utilization through nvidia-smi, which reports under 5 percent during decode. Both signals confirm the workload is bandwidth-starved, not compute-starved.

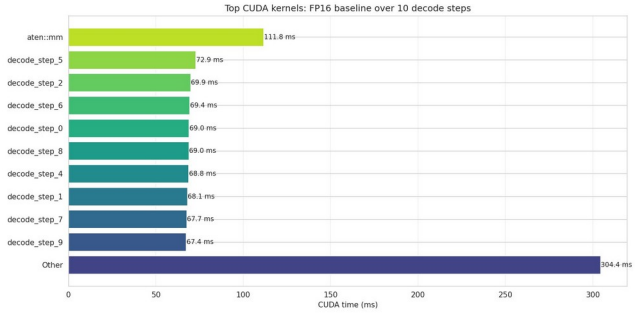

Fig. 5. PyTorch profiler kernel breakdown for 10 FP16 decode steps. aten::mm dominates at 111.8 ms, with the rest spread across many smaller operations for layer normalization, softmax, and elementwise operations.

E. Per-Task Variation

Acceptance rate varies meaningfully across prompt categories, even though the throughput numbers above pool over all categories. Code generation prompts achieve 16.2 tok/s at k=3 (acceptance 0.71), whereas conversational dialog prompts achieve only 13.1 tok/s (acceptance 0.59). Code is more syntactically constrained, with predictable token patterns (matching brackets, recurring keywords) that the small draft model handles well. Conversational dialog has higher entropy at every position, which is exactly what the draft struggles with. Multi-step reasoning prompts sit in the middle at 14.8 tok/s, factual question answering at 14.3 tok/s, and summarization at 13.9 tok/s. This per-task variation does

not change the headline result that speculative decoding underperforms FP16 on every category, but it does suggest that the right way to deploy speculative decoding in production is adaptively, with the speculation length chosen per prompt based on prompt features.

VI. CONCLUSION

We have presented a systems-level benchmark of LLM inference on a single NVIDIA A100 80GB GPU at batch size one, with detailed measurements of the FP16 baseline and a full sweep of speculative decoding configurations from k=1 through k=10. The headline finding is that speculative decoding, despite being widely promoted as a turn-key acceleration technique, does not produce a wall-clock speedup in this regime at any value of k. The diagnosis follows directly from a roofline analysis once the hardware constants are measured: the A100 ridge point sits at 156 FLOPs per byte, single-token decode operates at 2 FLOPs per byte, and the per-token time ratio between draft and target is only 1.22x because both models pay the same fixed kernel and attention overheads regardless of parameter count.

This work has limitations. All measurements are at batch size one; at larger batch sizes the workload shifts toward compute-bound, where speculative decoding has more room to help. We attempted INT8 and INT4 quantization to validate the roofline-predicted alternative, but were blocked by a binary compatibility issue between the installed bitsandbytes 0.43.1 and the Triton compiler that ships with Colab's PyTorch image (specifically, ModuleNotFoundError on the triton.ops namespace). Three workarounds were attempted; none resolved the issue cleanly within the project timeline. We treat this gap honestly rather than reporting predicted numbers as if they were measured. All measurements are also on a single A100 80GB; H100 has higher memory bandwidth and tensor core throughput, which would shift specific numbers but not the qualitative conclusions.

Future work includes a batch-size sweep to identify the crossover point at which speculative decoding becomes net-positive, revisiting quantization on a self-hosted environment with full control over the bitsandbytes/Triton stack, combining speculative decoding with quantization (a quantized target widens the draft-to-target time-ratio gap), and comparing against vLLM's continuous batching to characterize the throughput envelope of a production-grade serving system on the same hardware. The practical recommendation that emerges from this study is to run a roofline analysis before committing to any acceleration technique, and to match the technique to the regime: quantization for memory-bound workloads, speculative decoding for compute-bound workloads or for pipelines that fundamentally restructure the computation such as Medusa [7] and EAGLE [8].

VII. REFERENCES

[1] NVIDIA, "NVIDIA A100 Tensor Core GPU architecture," Whitepaper, 2020.

[2] R. Pope et al., "Efficiently scaling transformer inference," in Proc. Machine Learning and Systems, vol. 5, 2023, pp. 606-624.

[3] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in Proc. Int. Conf. Machine Learning, 2023, pp. 19274-19286.

[4] C. Chen et al., "Accelerating large language model decoding with speculative sampling," arXiv:2302.01318, 2023.

[5] T. Dettmers et al., "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in Proc. NeurIPS, vol. 35, 2022, pp. 30318-30332.

[6] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in Proc. NeurIPS, vol. 36, 2023.

[7] T. Cai et al., "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," in Proc. Int. Conf. Machine Learning, 2024.

[8] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE: Speculative sampling requires rethinking feature uncertainty," in Proc. Int. Conf. Machine Learning, 2024.

[9] M. Elhoushi et al., "LayerSkip: Enabling early exit inference and self-speculative decoding," in Proc. ACL, 2024.

[10] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," Communications of the ACM, vol. 52, no. 4, pp. 65-76, 2009.

[11] Z. Yuan et al., "LLM inference unveiled: Survey and roofline model insights," arXiv:2402.16363, 2024.

[12] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in Proc. SOSP, 2023, pp. 611-626.

[13] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in Proc. ICLR, 2023.